

Formal Verification of BVDOutbreakSize.jl

Simon Frost

Contents

Shared analytic vocabulary and trusted assumptions	1
Gauss-Legendre change-of-variable claims	5
Expected-death and export convolution claims	7
Gamma CDF derivative and rule claims	10
Cached CDF grid, trapezoid accumulation, and linear interpolation	12
Forecast and count-distribution algebra	14

```
import Mathlib.Analysis.Calculus.Deriv.Basic
import Mathlib.Analysis.SpecialFunctions.Exp
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Analysis.SpecialFunctions.Pow.Real
import Mathlib.MeasureTheory.Integral.IntervalIntegral.Basic
import Mathlib.Tactic.FieldSimp
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.NormNum
import Mathlib.Tactic.Positivity
import Mathlib.Tactic.Ring
```

Source: Foundations.lean

Shared analytic vocabulary and trusted assumptions

Most claims in BVDOutbreakSize.jl are algebraic identities once the continuous integral or special-function fact has been stated precisely. This module keeps those analytic facts explicit and named, so the rest of the proof tree can be checked without hiding which facts are trusted.

The axioms here are intentionally narrow:

- exact exponential integrals used by forecast/export formulas;
- the Gamma-convolution closed form used by expected_deaths(:: Gamma);
- Gamma CDF partial derivatives and Kummer expansion used by _gamma_cdf;
- exact substitution identities for the two quadrature maps.

They are mathematical specifications, not claims about Float64 arithmetic or fixed-order quadrature error.

```

noncomputable section

open scoped Interval

namespace BvdProofs

/-- A continuous-incidence trajectory `C(t)`. -/
abbrev Incidence := ℝ → ℝ

/-- A one-dimensional probability density. -/
abbrev Density := ℝ → ℝ

/-- A one-dimensional cumulative distribution function. -/
abbrev CDF := ℝ → ℝ

/-- The Gamma density with shape `α` and scale `θ`. -/
axiom gammaPDF : ℝ → ℝ → Density

/-- The Gamma CDF with shape `α` and scale `θ`. -/
axiom gammaCDF : ℝ → ℝ → CDF

/-- The Gamma moment-generating function, parameterized by shape and scale. -/
axiom gammaMGF : ℝ → ℝ → ℝ → ℝ

/-- Regularized lower incomplete Gamma `P(α,z)`. -/
axiom regularizedLowerGamma : ℝ → ℝ → ℝ

/-- Digamma function `ψ`. -/
axiom digamma : ℝ → ℝ

/-- The log-weighted lower-Gamma integral used in the shape derivative. -/
axiom lowerGammaLogIntegral : ℝ → ℝ → ℝ

/-- The Kummer-series sum  $\sum z^n / \Gamma(\alpha+n+1)$ . -/
axiom kummerPSeries : ℝ → ℝ → ℝ

/-- The Kummer-series sum with digamma factors. -/
axiom kummerDigammaSeries : ℝ → ℝ → ℝ

/-- CDF is the density integral for arguments on the nonnegative support. -/
axiom cdf_eq_density_integral
  (F : CDF) (f : Density) :
  (∀ x, F x = ∫ u in (0 : ℝ)..x, f u) →

```

```

     $\forall x, F x = \int u \text{ in } (0 : \mathbb{R})..x, f u$ 

/-- Exact affine substitution from `[lo, hi]` to `[-1, 1]`. -/
axiom affine_interval_integral
  (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (lo hi :  $\mathbb{R}$ ) :
    lo < hi  $\rightarrow$ 
      ( $\int s \text{ in } lo..hi, f s$ ) =
        ((hi - lo) / 2) *
           $\int u \text{ in } (-1 : \mathbb{R})..(1 : \mathbb{R}), f (((hi - lo) / 2) * (u + 1) + lo)$ 

/-- Exact clustered substitution used to cluster nodes toward the upper limit.

`expo` is real-valued in the Julia implementation. The theorem is stated over
the open-domain analytic map; endpoint singularities for `expo < 1` are part of
the standard improper-substitution theorem captured by this assumption. -/
axiom clustered_interval_integral
  (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (lo hi expo :  $\mathbb{R}$ ) :
    lo < hi  $\rightarrow$  0 < expo  $\rightarrow$ 
      ( $\int s \text{ in } lo..hi, f s$ ) =
         $\int u \text{ in } (-1 : \mathbb{R})..(1 : \mathbb{R}),$ 
          let v := (u + 1) / 2
          let span := hi - lo
          let d := span * v ^ expo
          f (hi - d) * (span * expo * v ^ (expo - 1) / 2)

/-- Exact integral of an exponential trajectory for nonzero growth. -/
axiom exp_interval_integral_nonzero
  (r lo hi :  $\mathbb{R}$ ) :
    r  $\neq$  0  $\rightarrow$ 
      ( $\int s \text{ in } lo..hi, \text{Real.exp } (r * s)$ ) =
        (Real.exp (r * hi) - Real.exp (r * lo)) / r

/-- Exact integral of a constant trajectory, the zero-growth limit. -/
axiom exp_interval_integral_zero
  (lo hi :  $\mathbb{R}$ ) :
    ( $\int s \text{ in } lo..hi, \text{Real.exp } ((0 : \mathbb{R}) * s)$ ) = hi - lo

/-- Gamma convolution closed form used by `expected_deaths(::Gamma)`.

The hypothesis `0 < 1 +  $\theta * r$ ` is the moment/CDF-domain condition for the
shifted Gamma term. -/
axiom gamma_death_convolution
  ( $\alpha \theta r T : \mathbb{R}$ ) :
    0 <  $\alpha \rightarrow$  0 <  $\theta \rightarrow$  0  $\leq$  T  $\rightarrow$  0 < 1 +  $\theta * r \rightarrow$ 
      ( $\int s \text{ in } (0 : \mathbb{R})..T,$ 

```

```

      Real.exp (r * s) * gammaPDF α θ (T - s)) =
      Real.exp (r * T) * gammaMGF α θ (-r) *
      gammaCDF α θ (T * (1 + θ * r))

/-- Gamma CDF is regularized lower Gamma at the scaled argument. -/
axiom gammaCDF_eq_regularized
  (α θ x : ℝ) :
  0 < α → 0 < θ → 0 ≤ x →
    gammaCDF α θ x = regularizedLowerGamma α (x / θ)

/-- Partial derivative of the Gamma CDF with respect to `x`. -/
axiom gammaCDF_hasDerivAt_x
  (α θ x : ℝ) :
  0 < α → 0 < θ → 0 < x →
    HasDerivAt (fun y ⇒ gammaCDF α θ y) (gammaPDF α θ x) x

/-- Partial derivative of the Gamma CDF with respect to the scale `θ`. -/
axiom gammaCDF_hasDerivAt_theta
  (α θ x : ℝ) :
  0 < α → 0 < θ → 0 < x →
    HasDerivAt (fun η ⇒ gammaCDF α η x)
      (-(x / θ) * gammaPDF α θ x) θ

/-- Partial derivative of regularized lower Gamma with respect to shape. -/
axiom regularizedLowerGamma_hasDerivAt_alpha
  (α z : ℝ) :
  0 < α → 0 < z →
    HasDerivAt (fun a ⇒ regularizedLowerGamma a z)
      (-digamma α * regularizedLowerGamma α z + lowerGammaLogIntegral α z) α

/-- Kummer expansion for the regularized lower Gamma. -/
axiom regularizedLowerGamma_kummer
  (α z : ℝ) :
  0 < α → 0 < z →
    regularizedLowerGamma α z =
      z ^ α * Real.exp (-z) * kummerPSeries α z

/-- Shape derivative from the differentiated Kummer expansion. -/
axiom regularizedLowerGamma_kummer_derivative
  (α z : ℝ) :
  0 < α → 0 < z →
    HasDerivAt (fun a ⇒ regularizedLowerGamma a z)
      (Real.log z * regularizedLowerGamma α z -
        z ^ α * Real.exp (-z) * kummerDigammaSeries α z) α

```

```

/-- Digamma recurrence used by the iterative series implementation. -/
axiom digamma_succ
  (x : ℝ) :
    x ≠ 0 → digamma (x + 1) = digamma x + 1 / x

end BvdProofs

```

```
import BvdProofs.Foundations
```

Source: Quadrature.lean

Gauss-Legendre change-of-variable claims

The Julia code integrates on $[-1, 1]$ and maps those nodes to the target interval. This file proves the endpoint and order facts for the maps, and connects the kernels to the exact substitution assumptions in Foundations.lean.

```

noncomputable section

open scoped Interval

namespace BvdProofs

/-- Half the interval width used as the affine Jacobian. -/
def halfwidth (lo hi : ℝ) : ℝ := (hi - lo) / 2

/-- The affine map from Gauss-Legendre reference nodes to `[lo, hi]`. -/
def affineNode (lo hi u : ℝ) : ℝ := halfwidth lo hi * (u + 1) + lo

/-- Reference-coordinate normalization for the clustered map. -/
def clusterV (u : ℝ) : ℝ := (u + 1) / 2

/-- Distance back from the upper endpoint in the clustered map. -/
def clusterDistance (span expo u : ℝ) : ℝ := span * clusterV u ^ expo

/-- Target point for clustered quadrature nodes. -/
def clusterPoint (lo hi expo u : ℝ) : ℝ :=
  hi - clusterDistance (hi - lo) expo u

/-- Positive Jacobian `dd/du`; the sign flip is handled by integrating in distance. -/
def clusterJacobian (span expo u : ℝ) : ℝ :=
  span * expo * clusterV u ^ (expo - 1) / 2

theorem halfwidth_pos {lo hi : ℝ} (h : lo < hi) : 0 < halfwidth lo hi := by
  dsimp [halfwidth]
  linarith

```

```

theorem affineNode_neg_one (lo hi : ℝ) :
  affineNode lo hi (-1) = lo := by
  simp [affineNode, halfwidth]

theorem affineNode_one (lo hi : ℝ) :
  affineNode lo hi 1 = hi := by
  simp [affineNode, halfwidth]
  ring

theorem affineNode_midpoint (lo hi : ℝ) :
  affineNode lo hi 0 = (lo + hi) / 2 := by
  simp [affineNode, halfwidth]
  ring

theorem affineNode_sub_lo (lo hi u : ℝ) :
  affineNode lo hi u - lo = halfwidth lo hi * (u + 1) := by
  simp [affineNode]

theorem affineNode_in_interval {lo hi u : ℝ}
  (hlohi : lo ≤ hi) (hu_lower : -1 ≤ u) (hu_upper : u ≤ 1) :
  lo ≤ affineNode lo hi u ∧ affineNode lo hi u ≤ hi := by
  constructor
  · dsimp [affineNode, halfwidth]
    have hwidth : 0 ≤ (hi - lo) / 2 := by linarith
    have hu : 0 ≤ u + 1 := by linarith
    nlinarith [mul_nonneg hwidth hu]
  · dsimp [affineNode, halfwidth]
    have hwidth : 0 ≤ (hi - lo) / 2 := by linarith
    have hu : u + 1 ≤ 2 := by linarith
    have hmul : ((hi - lo) / 2) * (u + 1) ≤ ((hi - lo) / 2) * 2 :=
      mul_le_mul_of_nonneg_left hu hwidth
    nlinarith

theorem affine_kernel_matches_integral
  (f : ℝ → ℝ) {lo hi : ℝ} (hlohi : lo < hi) :
  (∫ s in lo..hi, f s) =
    halfwidth lo hi *
    ∫ u in (-1 : ℝ)..(1 : ℝ), f (affineNode lo hi u) := by
  simp [halfwidth, affineNode] using affine_interval_integral f lo hi hlohi

theorem clusterV_neg_one : clusterV (-1) = 0 := by
  simp [clusterV]

theorem clusterV_one : clusterV 1 = 1 := by
  simp [clusterV]

```

```

theorem clusterV_in_unit {u : ℝ}
  (hu_lower : -1 ≤ u) (hu_upper : u ≤ 1) :
  0 ≤ clusterV u ∧ clusterV u ≤ 1 := by
  constructor <;> dsimp [clusterV] <;> linarith

theorem clusterPoint_one (lo hi expo : ℝ) :
  clusterPoint lo hi expo 1 = lo := by
  simp [clusterPoint, clusterDistance, clusterV]

theorem clusterPoint_neg_one {lo hi expo : ℝ} (hexpo : 0 < expo) :
  clusterPoint lo hi expo (-1) = hi := by
  have hzero : (0 : ℝ) ^ expo = 0 := Real.zero_rpow (ne_of_gt hexpo)
  simp [clusterPoint, clusterDistance, clusterV, hzero]

theorem clusterJacobian_nonneg {span expo u : ℝ}
  (hspan : 0 ≤ span) (hexpo : 0 ≤ expo)
  (hu : 0 ≤ clusterV u ^ (expo - 1)) :
  0 ≤ clusterJacobian span expo u := by
  dsimp [clusterJacobian]
  positivity

theorem clustered_kernel_matches_integral
  (f : ℝ → ℝ) {lo hi expo : ℝ}
  (hlohi : lo < hi) (hexpo : 0 < expo) :
  (∫ s in lo..hi, f s) =
    ∫ u in (-1 : ℝ)..(1 : ℝ),
      f (clusterPoint lo hi expo u) *
        clusterJacobian (hi - lo) expo u := by
  simpa [clusterPoint, clusterDistance, clusterJacobian, clusterV]
  using clustered_interval_integral f lo hi expo hlohi hexpo

end BvdProofs

import BvdProofs.Quadrature

```

Source: Expectations.lean

Expected-death and export convolution claims

This module gives Lean names to the continuous expectations documented in `BVDOutbreakSize.jl`. The theorems below prove the package formulas from these definitions and from the named Gamma-convolution assumption.

```
noncomputable section
```

```
open scoped Interval
```

```

namespace BvdProofs

/-- Cumulative deaths from a single seed under exponential growth and delay density. -/
def expectedDeaths (CFR r T : ℝ) (density : Density) : ℝ :=
  CFR * ∫ s in (0 : ℝ)..T, Real.exp (r * s) * density (T - s)

/-- Expected deaths using a Gamma onset-to-death delay. -/
def expectedDeathsGamma (CFR r T α θ : ℝ) : ℝ :=
  CFR * Real.exp (r * T) * gammaMGF α θ (-r) *
    gammaCDF α θ (T * (1 + θ * r))

theorem expectedDeaths_eq_convolution
  (CFR r T : ℝ) (density : Density) :
    expectedDeaths CFR r T density =
      CFR * ∫ s in (0 : ℝ)..T, Real.exp (r * s) * density (T - s) := by
    rfl

theorem expectedDeathsGamma_eq_closed_form
  {CFR r T α θ : ℝ}
  (hα : 0 < α) (hθ : 0 < θ) (hT : 0 ≤ T) (hdom : 0 < 1 + θ * r) :
    expectedDeaths CFR r T (gammaPDF α θ) =
      expectedDeathsGamma CFR r T α θ := by
    unfold expectedDeaths expectedDeathsGamma
    rw [gamma_death_convolution α θ r T hα hθ hT hdom]
    ring

/-- CDF built as an integral of its density, the generic AD-friendly path. -/
def cdfFromDensity (density : Density) : CDF :=
  fun x => ∫ u in (0 : ℝ)..x, density u

theorem cdfFromDensity_eq_integral (density : Density) (x : ℝ) :
  cdfFromDensity density x = ∫ u in (0 : ℝ)..x, density u := by
  rfl

/-- Deaths-among-exports convolution with an arbitrary delay CDF. -/
def exportsDeathsIntegral (C : Incidence) (F : CDF) (lo hi T : ℝ) : ℝ :=
  ∫ s in lo..hi, C s * F (T - s)

theorem exportsDeathsIntegral_with_density_cdf
  (C : Incidence) (density : Density) (lo hi T : ℝ) :
    exportsDeathsIntegral C (cdfFromDensity density) lo hi T =
      ∫ s in lo..hi, C s * (∫ u in (0 : ℝ)..(T - s), density u) := by
    rfl

theorem exportsDeathsIntegral_gamma

```



```

(C : Incidence) (lo hi T α θ : ℝ) :
  exportsDeathsIntegral C (gammaCDF α θ) lo hi T =
    ∫ s in lo..hi, C s * gammaCDF α θ (T - s) := by
rfl

/-- Detection-window lower endpoint. -/
def windowStart (t window : ℝ) : ℝ := max (t - window) 0

theorem windowStart_nonneg (t window : ℝ) :
  0 ≤ windowStart t window := by
  exact le_max_right (t - window) 0

theorem windowStart_le_t {t window : ℝ}
  (ht : 0 ≤ t) (hw : 0 ≤ window) :
  windowStart t window ≤ t := by
  dsimp [windowStart]
  exact max_le (by linarith) ht

/-- Expected detected exports over the detection window. -/
def expectedExports (C : Incidence) (p q t window : ℝ) : ℝ :=
  p * q * ∫ s in windowStart t window..t, C s

theorem expectedExports_eq_window_integral
  (C : Incidence) (p q t window : ℝ) :
  expectedExports C p q t window =
    p * q * ∫ s in windowStart t window..t, C s := by
rfl

/-- Expected deaths among detected exports over the detection window. -/
def expectedExportDeaths
  (C : Incidence) (F : CDF) (CFR p q t window : ℝ) : ℝ :=
  CFR * p * q * exportsDeathsIntegral C F (windowStart t window) t t

theorem expectedExportDeaths_eq_window_convolution
  (C : Incidence) (F : CDF) (CFR p q t window : ℝ) :
  expectedExportDeaths C F CFR p q t window =
    CFR * p * q *
    ∫ s in windowStart t window..t, C s * F (t - s) := by
rfl

/-- Future deaths among infections that already occurred by `T`. -/
def committedDeaths (CFR r T : ℝ) (F : CDF) : ℝ :=
  CFR * ∫ s in (0 : ℝ)..T, r * Real.exp (r * s) * (1 - F (T - s))

theorem committedDeaths_eq_counterfactual_integral

```

```

    (CFR r T : ℝ) (F : CDF) :
      committedDeaths CFR r T F =
        CFR * ∫ s in (0 : ℝ)..T, r * Real.exp (r * s) * (1 - F (T - s)) := by
      rfl

/-- Projected total under no-onward transmission. -/
def totalProjectedDeaths (obsDeaths deltaDeaths : ℝ) : ℝ :=
  obsDeaths + deltaDeaths

theorem totalProjectedDeaths_eq_obs_plus_delta
  (obsDeaths deltaDeaths : ℝ) :
    totalProjectedDeaths obsDeaths deltaDeaths = obsDeaths + deltaDeaths := by
  rfl

end BvdProofs

```

```
import BvdProofs.Expectations
```

Source: GammaCDF.lean

Gamma CDF derivative and rrule claims

src/gamma_cdf.jl attaches a ChainRules/Mooncake reverse rule to a three-argument Gamma CDF wrapper. This module states the analytic partials and proves the scalar pullback algebra used by the rule.

```

noncomputable section

namespace BvdProofs

/-- Shape derivative formula from the integral representation. -/
def gammaShapePartial (α z : ℝ) : ℝ :=
  -digamma α * regularizedLowerGamma α z + lowerGammaLogIntegral α z

/-- Shape derivative formula from the differentiated Kummer expansion. -/
def gammaShapePartialKummer (α z : ℝ) : ℝ :=
  Real.log z * regularizedLowerGamma α z -
  z ^ α * Real.exp (-z) * kummerDigammaSeries α z

/-- Scale derivative formula under shape-scale parameterization. -/
def gammaScalePartial (α θ x : ℝ) : ℝ :=
  -(x / θ) * gammaPDF α θ x

/-- Argument derivative of a CDF is its density. -/
def gammaArgumentPartial (α θ x : ℝ) : ℝ :=
  gammaPDF α θ x

```

```

theorem gammaCDF_x_partial
  {α θ x : ℝ} (hα : 0 < α) (hθ : 0 < θ) (hx : 0 < x) :
  HasDerivAt (fun y ⇒ gammaCDF α θ y)
    (gammaArgumentPartial α θ x) x := by
  simpa [gammaArgumentPartial] using
    gammaCDF_hasDerivAt_x α θ x hα hθ hx

theorem gammaCDF_theta_partial
  {α θ x : ℝ} (hα : 0 < α) (hθ : 0 < θ) (hx : 0 < x) :
  HasDerivAt (fun η ⇒ gammaCDF α η x)
    (gammaScalePartial α θ x) θ := by
  simpa [gammaScalePartial] using
    gammaCDF_hasDerivAt_theta α θ x hα hθ hx

theorem regularizedLowerGamma_alpha_partial
  {α z : ℝ} (hα : 0 < α) (hz : 0 < z) :
  HasDerivAt (fun a ⇒ regularizedLowerGamma a z)
    (gammaShapePartial α z) α := by
  simpa [gammaShapePartial] using
    regularizedLowerGamma_hasDerivAt_alpha α z hα hz

theorem regularizedLowerGamma_alpha_partial_kummer
  {α z : ℝ} (hα : 0 < α) (hz : 0 < z) :
  HasDerivAt (fun a ⇒ regularizedLowerGamma a z)
    (gammaShapePartialKummer α z) α := by
  simpa [gammaShapePartialKummer] using
    regularizedLowerGamma_kummer_derivative α z hα hz

theorem regularizedLowerGamma_kummer_formula
  {α z : ℝ} (hα : 0 < α) (hz : 0 < z) :
  regularizedLowerGamma α z =
    z ^ α * Real.exp (-z) * kummerPSeries α z := by
  exact regularizedLowerGamma_kummer α z hα hz

/-- Scalar pullback returned by the ChainRules rrule. -/
def gammaCdfPullback
  (α θ x seed : ℝ) : ℝ × ℝ × ℝ :=
  (seed * gammaShapePartialKummer α (x / θ),
   seed * gammaScalePartial α θ x,
   seed * gammaArgumentPartial α θ x)

theorem gammaCdfPullback_components
  (α θ x seed : ℝ) :
  gammaCdfPullback α θ x seed =
    (seed * gammaShapePartialKummer α (x / θ),

```

```

      seed * (-(x /  $\theta$ ) * gammaPDF  $\alpha$   $\theta$  x),
      seed * gammaPDF  $\alpha$   $\theta$  x) := by
    rfl

theorem digamma_recurrence_for_series_step
  { $\alpha$  :  $\mathbb{R}$ } {n :  $\mathbb{N}$ } (h :  $\alpha + n \neq 0$ ) :
  digamma ( $\alpha + n + 1$ ) = digamma ( $\alpha + n$ ) + 1 / ( $\alpha + n$ ) := by
  simpa [add_assoc] using digamma_succ ( $\alpha + n$ ) h

end BvdProofs

import BvdProofs.Expectations

```

Source: Interpolation.lean

Cached CDF grid, trapezoid accumulation, and linear interpolation

ExportDeathDelay precomputes a cumulative trapezoid approximation to the delay CDF and then linearly interpolates it. These are approximation claims, not exact equalities to the analytic CDF. The exact Lean results below cover the deterministic algebra and clamp behavior of that approximation layer.

```

noncomputable section

namespace BvdProofs

/-- One cumulative-trapezoid update. -/
def trapezoidStep (Fprev fprev fcur dx :  $\mathbb{R}$ ) :  $\mathbb{R}$  :=
  Fprev + (fprev + fcur) * dx / 2

theorem trapezoidStep_increment
  (Fprev fprev fcur dx :  $\mathbb{R}$ ) :
  trapezoidStep Fprev fprev fcur dx - Fprev =
  (fprev + fcur) * dx / 2 := by
  simp [trapezoidStep]

theorem trapezoidStep_monotone
  {Fprev fprev fcur dx :  $\mathbb{R}$ }
  (hfprev :  $0 \leq fprev$ ) (hfcur :  $0 \leq fcur$ ) (hdx :  $0 \leq dx$ ) :
  Fprev  $\leq$  trapezoidStep Fprev fprev fcur dx := by
  dsimp [trapezoidStep]
  have hprod :  $0 \leq (fprev + fcur) * dx / 2$  := by positivity
  linarith

/-- Linear interpolation in a grid cell. `frac=0` gives the left endpoint,
`frac=1` gives the right endpoint. -/

```

```

def linearInterp (left right frac : ℝ) : ℝ :=
  left + frac * (right - left)

theorem linearInterp_left (left right : ℝ) :
  linearInterp left right 0 = left := by
  simp [linearInterp]

theorem linearInterp_right (left right : ℝ) :
  linearInterp left right 1 = right := by
  simp [linearInterp]

theorem linearInterp_convex_form (left right frac : ℝ) :
  linearInterp left right frac =
    (1 - frac) * left + frac * right := by
  simp [linearInterp]
  ring

theorem linearInterp_between
  {left right frac : ℝ}
  (hle : left ≤ right) (h0 : 0 ≤ frac) (h1 : frac ≤ 1) :
  left ≤ linearInterp left right frac ∧
  linearInterp left right frac ≤ right := by
  constructor
  · dsimp [linearInterp]
    have hdiff : 0 ≤ right - left := by linarith
    have hterm : 0 ≤ frac * (right - left) := mul_nonneg h0 hdiff
    linarith
  · rw [linearInterp_convex_form]
    have hleft : (1 - frac) * left ≤ (1 - frac) * right := by
      exact mul_le_mul_of_nonneg_left hle (by linarith)
    have hright : frac * right ≤ frac * right := le_rfl
    have hsum : (1 - frac) * left + frac * right ≤
      (1 - frac) * right + frac * right := add_le_add hleft hright
    calc
      (1 - frac) * left + frac * right
        ≤ (1 - frac) * right + frac * right := hsum
      _ = right := by ring

/-- The branch structure of `_cdf_to` : zero before support, flat after `gmax`,
and otherwise the cell interpolant. -/
def cachedCDF (gmax last : ℝ) (interp : ℝ → ℝ) (y : ℝ) : ℝ :=
  if y ≤ 0 then 0 else if gmax ≤ y then last else interp y

theorem cachedCDF_nonpositive
  (gmax last : ℝ) (interp : ℝ → ℝ) {y : ℝ} (hy : y ≤ 0) :

```

```

    cachedCDF gmax last interp y = 0 := by
    simp [cachedCDF, hy]

theorem cachedCDF_past_gmax
  {gmax last y : ℝ} (interp : ℝ → ℝ)
  (hy0 : ¬ y ≤ 0) (hyg : gmax ≤ y) :
  cachedCDF gmax last interp y = last := by
  simp [cachedCDF, hy0, hyg]

theorem cachedCDF_inside
  {gmax last y : ℝ} (interp : ℝ → ℝ)
  (hy0 : ¬ y ≤ 0) (hyg : ¬ gmax ≤ y) :
  cachedCDF gmax last interp y = interp y := by
  simp [cachedCDF, hy0, hyg]

end BvdProofs

import BvdProofs.GammaCDF

```

Source: Forecast.lean

Forecast and count-distribution algebra

The forecast path reuses the expected-deaths formula, integrates exponential growth over the export detection window in closed form, and converts a desired negative-binomial mean to the `Distributions.jl` success-probability parameter.

```

noncomputable section

open scoped Interval

namespace BvdProofs

/-- Closed-form export mean for exponential incidence over `[lo, hi]`. -/
def forecastExportsClosed (p q r lo hi : ℝ) : ℝ :=
  p * q * (Real.exp (r * hi) - Real.exp (r * lo)) / r

/-- Zero-growth limit of the export mean. -/
def forecastExportsZeroGrowth (p q lo hi : ℝ) : ℝ :=
  p * q * (hi - lo)

theorem exponential_export_integral_closed
  {p q r lo hi : ℝ} (hr : r ≠ 0) :
  p * q * (∫ s in lo..hi, Real.exp (r * s)) =
  forecastExportsClosed p q r lo hi := by

```

```

rw [exp_interval_integral_nonzero r lo hi hr]
simp [forecastExportsClosed]
ring

theorem exponential_export_integral_zero_growth
  (p q lo hi : ℝ) :
  p * q * (∫ s in lo..hi, Real.exp ((0 : ℝ) * s)) =
    forecastExportsZeroGrowth p q lo hi := by
  rw [exp_interval_integral_zero lo hi]
  rfl

theorem forecast_deaths_reuses_expectedDeathsGamma
  (CFR r Th α θ : ℝ) :
  expectedDeathsGamma CFR r Th α θ =
    CFR * Real.exp (r * Th) * gammaMGF α θ (-r) *
    gammaCDF α θ (Th * (1 + θ * r)) := by
  rfl

/-- Success-probability parameter for a negative-binomial count with mean `μ`
and dispersion/shape `k`, under the failures-before-`k`-successes convention. -/
def nbSuccessProb (k μ : ℝ) : ℝ := k / (k + μ)

theorem nbSuccessProb_pos {k μ : ℝ}
  (hk : 0 < k) (hμ : 0 ≤ μ) :
  0 < nbSuccessProb k μ := by
  dsimp [nbSuccessProb]
  positivity

theorem nbSuccessProb_lt_one {k μ : ℝ}
  (hk : 0 < k) (hμ : 0 < μ) :
  nbSuccessProb k μ < 1 := by
  dsimp [nbSuccessProb]
  have hden : 0 < k + μ := by positivity
  rw [div_lt_one hden]
  linarith

theorem negativeBinomial_mean_from_prob {k μ : ℝ}
  (hk : 0 < k) (hμ : 0 < μ) :
  k * (1 - nbSuccessProb k μ) / nbSuccessProb k μ = μ := by
  dsimp [nbSuccessProb]
  have hden : k + μ ≠ 0 := by positivity
  have hk_ne : k ≠ 0 := ne_of_gt hk
  field_simp [hden, hk_ne]
  linarith

```

```

/-- New reported counts are floored at zero. -/
def newCount (cum obs : ℝ) : ℝ := max (cum - obs) 0

theorem newCount_nonnegative (cum obs : ℝ) :
  0 ≤ newCount cum obs := by
  exact le_max_right (cum - obs) 0

theorem newCount_eq_difference_when_ge {cum obs : ℝ}
  (h : obs ≤ cum) :
  newCount cum obs = cum - obs := by
  dsimp [newCount]
  exact max_eq_left (by linarith)

end BvdProofs

```